NAMES:UWAJENEZA Beatrice

RegNO:223028059

Mobile Computing Assignment

Lab 5: Offline Posts Manager (SQLite)

1. Dependencies Used & Handling Database Exceptions

OFFLINE POSTMANAGER- HANDWRITTEN ANSWERS.

1. Dependences used and why Sqlite is Necessary

Dependencies Used:

1. Sqlite: ^2.3.0
- primarly sqlite database Api for flutter
- provides Complete CRUD (Create, Read, update, Delete) operations
- Handles database creation, opening, and query Execution.
- platform-specific Implementation for Android and iOS

2. path: ^1.8.3
- Used for Safe path Construction across different platforms.
- Joins directory paths Correctly (window uses, macos/Linux uses/)
- Ensures database file is created in the Correct application documents directory

3. intl: ^0.18.1
- Internationalization package for date/time formatting.
- provides Constistent date display across different locales.
- Used to format created_at and updated_at timestamps.

Why Sqlite is Necessary for Local Data Storage

Offline Capability:
- Sqlite Stores all data locally on the device's internal Storage
- No internet Connection required to access, Create edit or delete posts.
- perfect for media Company Staff working in Areas with poor Connectivity

Data persistence:
- Data remains even after app is closed and reopened.
- Unlike in-memory storage, SQLite persists across app restarts.
- Device reboots do not affect Stored data.

Structured Data:
- provides a relational database structure with table, columns and data types.

- Enforces data integrity Through constraints (NOT NULL, PRIMARY KEY, etc).
- Allows complex queries with filtering, sorting and searching

Performance:

- Lightweight embedded database engine
- optimized for Mobile device with minimal memory footprint.
- Can handle Thousands of posts efficiently without performance degradation.

ACID Compliances

Atomicity: Transactions complete fully or not at all

Consistency: Database constraints prevent invalid data.

Isolation: Concurrent operations don't interfere

Durability: Committed data Survives Systems Crashes.

SQL Standard:

- Uses Standard SQL syntax queries
- Easy to learn and Implement.
- Familiar to developers with SQL experience

2. How to handle database exceptions.

1. Database Not Initialized handling

problem:

Database not yet created or opened

Solution

```
if (_database != null) return _database!;
_database = await _InitDB('post.db');
```

Ensures that database is initialized before use

2. Insert/update/delete errors.

use try-catch

```
try {
  await db.insert('Posts', Post);
} catch (e) {
  Print('Insert error: $e');
}
```

- prevents app crash
- Helps debugging

3. Invalid and Corrupted data
Example handling

```
if( title.isEmpty || body.isEmpty){
  Throw Exception ("Invalid data"); }
```

- Prevents bad data Entering database

3. Explain SpLite in flutter

1. Database vs table

Database	table
- * file that stores data	- a structure inside database (rows + Columns)
- Can Contain multiple table	- our app has one table
- Managed by DatabaseHelper class as singleton	- Structured Collection of related data within database

Example

```
CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    body TEXT,
);
```

2. CRUD OPERATIONS

- CREATE (Insert data)

```
await db.Insert('posts', {
  'title': title,
  'body': body,
});
```

- adds new record to table
- Returns auto generated ID
- Validate data before Insertion.

- READ (Retrieve data

final list < map < string, dynamic >> posts = await db.query('posts');

- Retrieves All record
- Can filter with where clause
- ~~Update~~
- Can sort with order by ORDER By
- Returns List of post objects.

- Update (UPDATE)

```
Future <int> updatePost (Post Post) async {
database db = await database;
return  await db.update (
  'posts',
   Post.toMap(),
   where : 'id = ?',
   Where Args : [Post.Id];
   );
}
```

* Modify existing record * Returns number of rows
* Can not recover deleted data (permanently)

- How flutter Interacts with the database Asynchronously

Flutter uses asynchronous programming (async/wait)

Example : future < list < Map < String, dynamic >>> getposts()
```
async {
    final db = await DatabaseHelper.instance.database;
    return await db.query ('Posts'); }
```

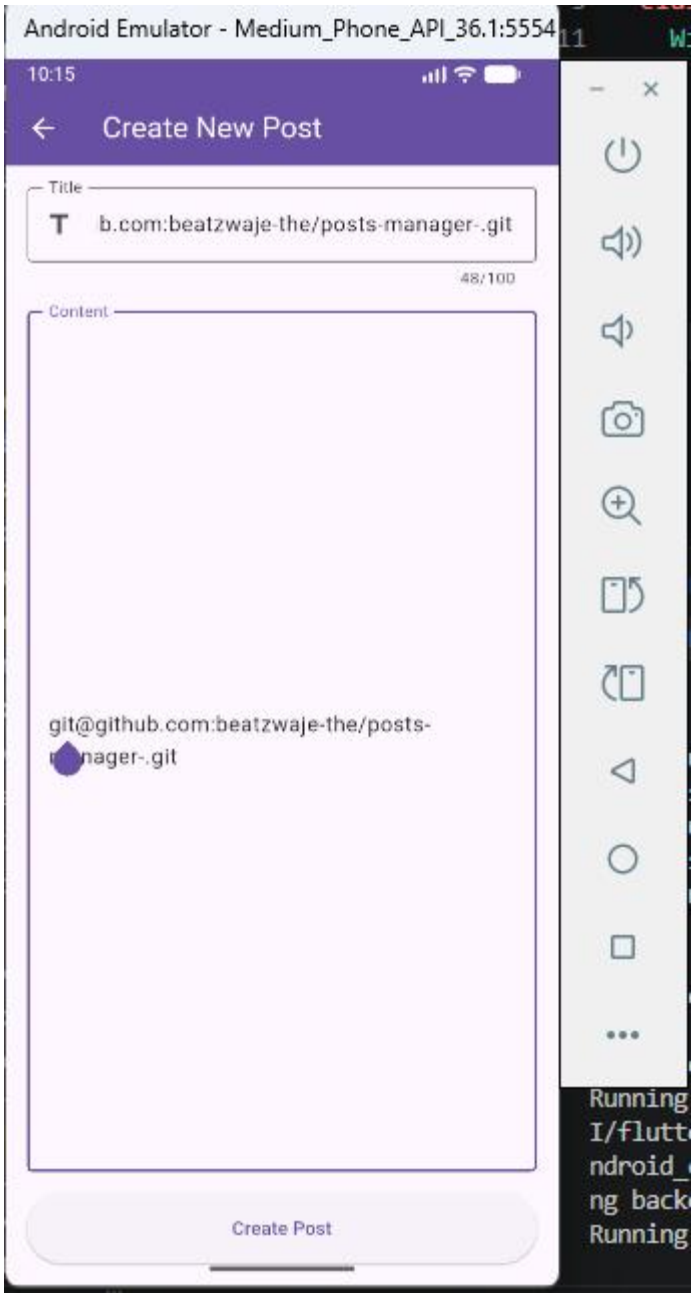
Why async ?

- Database operation take time
- prevents UI freezing
- Ensures smooth user experience.

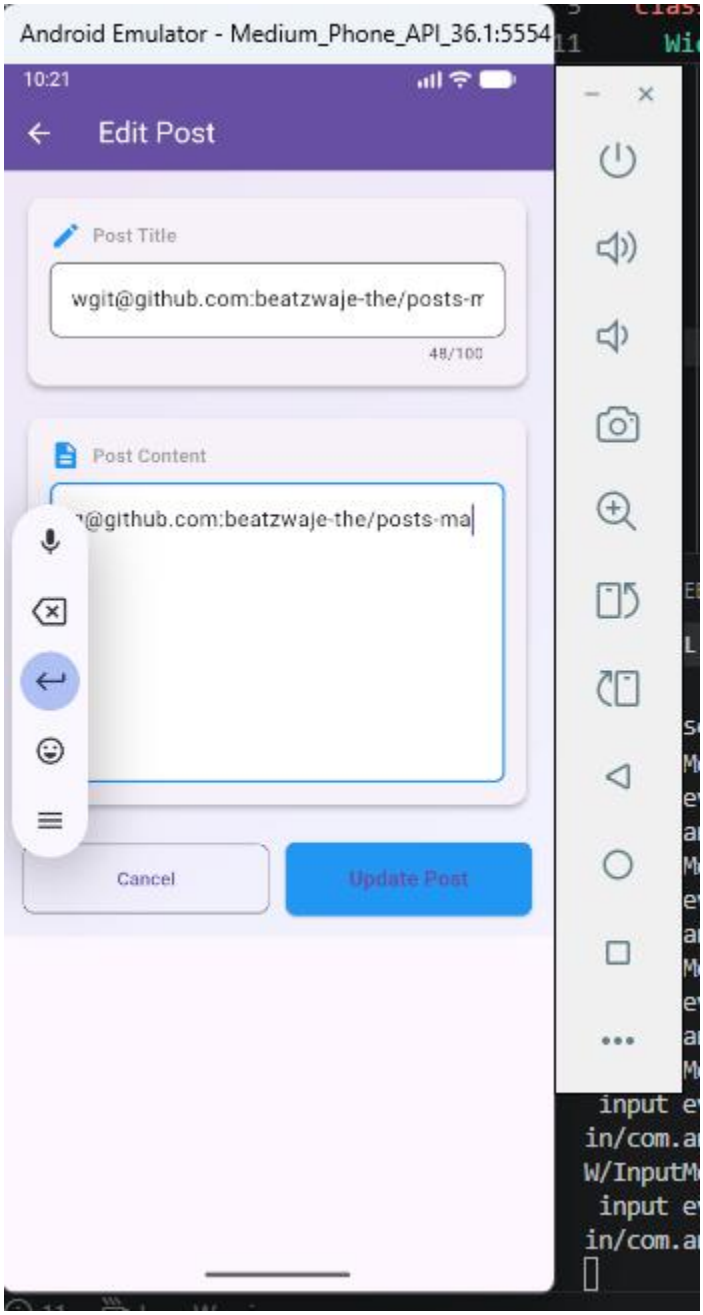
5. Features Implemented

- View posts locally

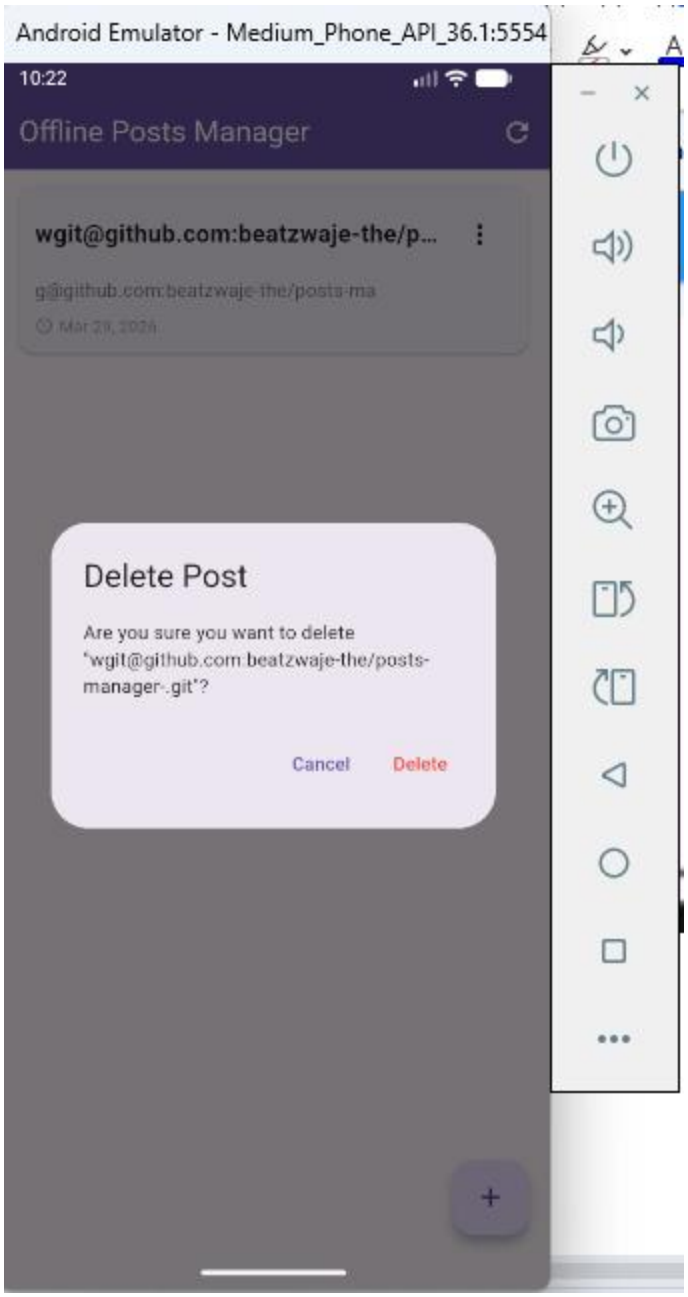
- Add new post


Android Emulator - Medium_Phone_API_36.1:5554
10:15
Create New Post
Title
b.com:beatzwaje-the/posts-manager-.git
48/100
Content
git@github.com:beatzwaje-the/posts-
nager-.git
Create Post
Running
I/flutt
ndroid_
ng back
Running

- Edit post


Android Emulator - Medium_Phone_API_36.1:5554
10:21
Edit Post
Post Title
wgit@github.com:beatzwaje-the/posts-m
48/100
Post Content
@github.com:beatzwaje-the/posts-ma
Cancel
Update Post

- Delete post


Android Emulator - Medium_Phone_API_36.1:5554
10:22
Offline Posts Manager
wgit@github.com:beatzwaje-the/p...
g@github.com:beatzwaje-the/posts-ma
Delete Post
Are you sure you want to delete "wgit@github.com:beatzwaje-the/posts-manager-.git"?
Cancel
Delete